Natural Language Processing
09. Sequence to Sequence Models and Attention

Juan José Alegría

May 13, 2025

Table of contents

RNN usage-patterns

RNN usage-patterns

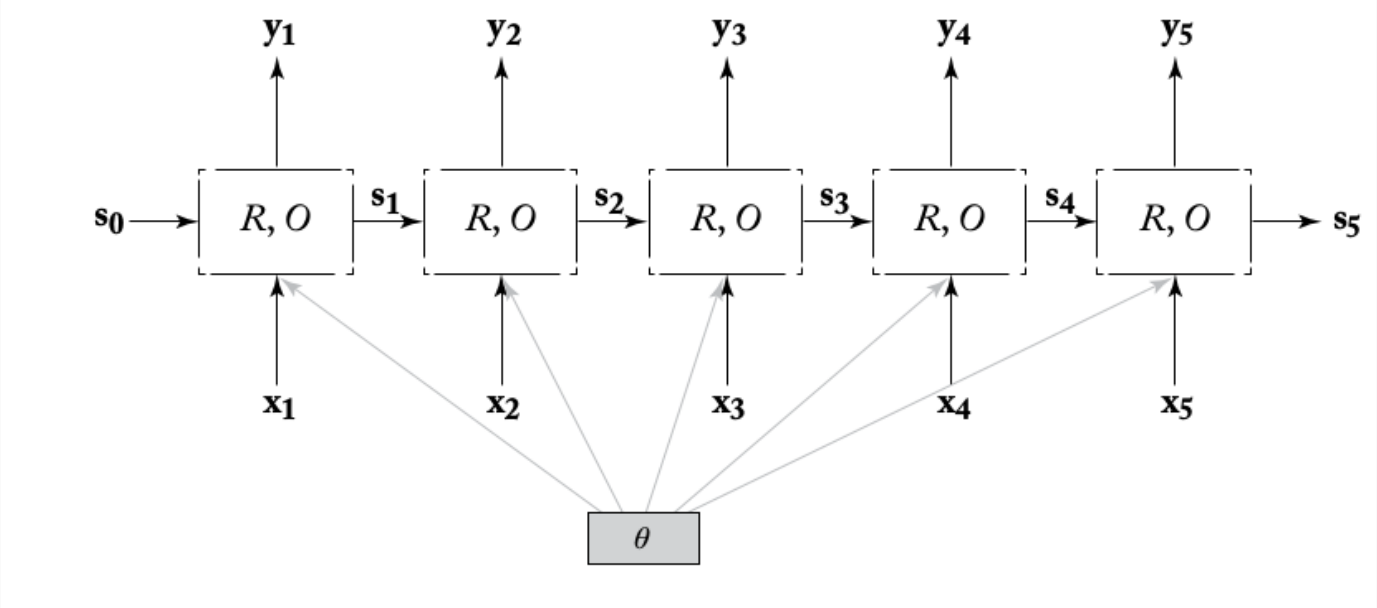


Figure 1: Unrolled RNN

- We have seen that RNNs produce multiple outputs: one for each token
- How can we use them for NLP tasks?
- It turns out that there are different patterns that we can use, depending on the task we are trying to perform.

RNN usage-patterns: Acceptor

- A common RNN usage-pattern is the acceptor.
- This pattern is used for text classification.
- The supervision signal is only based on the final output vector $\vec{y}_n$.
- The RNN's output vector $\vec{y}_n$ is fed into a fully connected layer or an MLP, which produce a prediction.
- The error gradients are then backpropagated through the rest of the sequence.
- The loss can take any familiar form: cross entropy, hinge, etc.

RNN usage-patterns: Acceptor

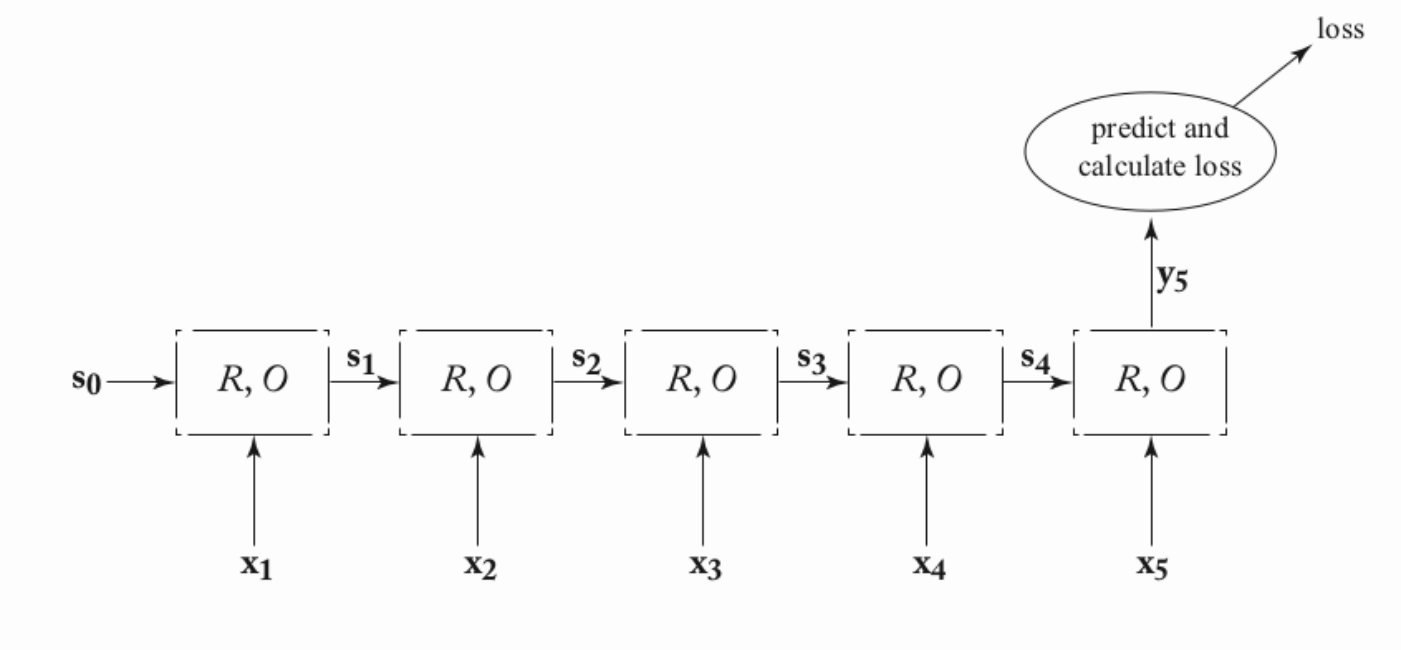


Figure 2: Acceptor RNN training graph.

- Example 1: an RNN that reads the characters of a word one by one and then use the final state to predict the part-of-speech of that word.
- Example 2: an RNN that reads in a sentence and, based on the final state decides if it conveys positive or negative sentiment.

RNN usage-patterns: Transducer

- Another option is to treat the RNN as a transducer, producing an output $\hat{t}_i$ for each input it reads in.
- This pattern is very handy for sequence labeling tasks (e.g, POS tagging, NER).
- We compute a local loss signal (e.g., cross-entropy) $L_{\text{local}}(\hat{t}_i, t_i)$ for each of the outputs $\hat{t}_i$ based on a true label t_i.
- The loss for unrolled sequence will then be: $L(\hat{t}_{i:n}, t_{i:n}) = \sum_i L_{\text{local}}(\hat{t}_i, t_i)$, or using another combination rather than a sum such as an average or a weighted average

RNN usage-patterns: Transducer

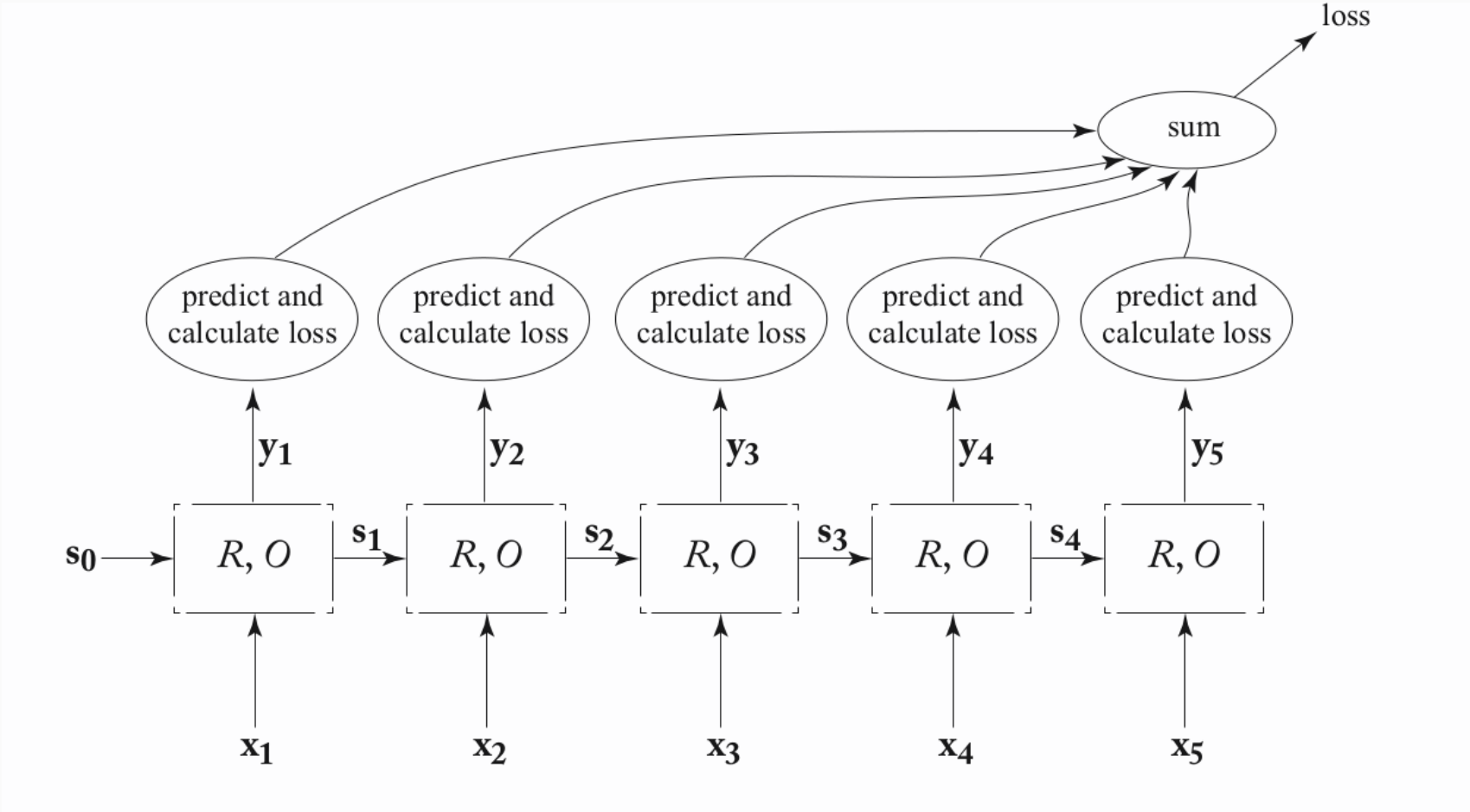


Figure 3: Transducer RNN training graph.

RNN usage-patterns: Transducer

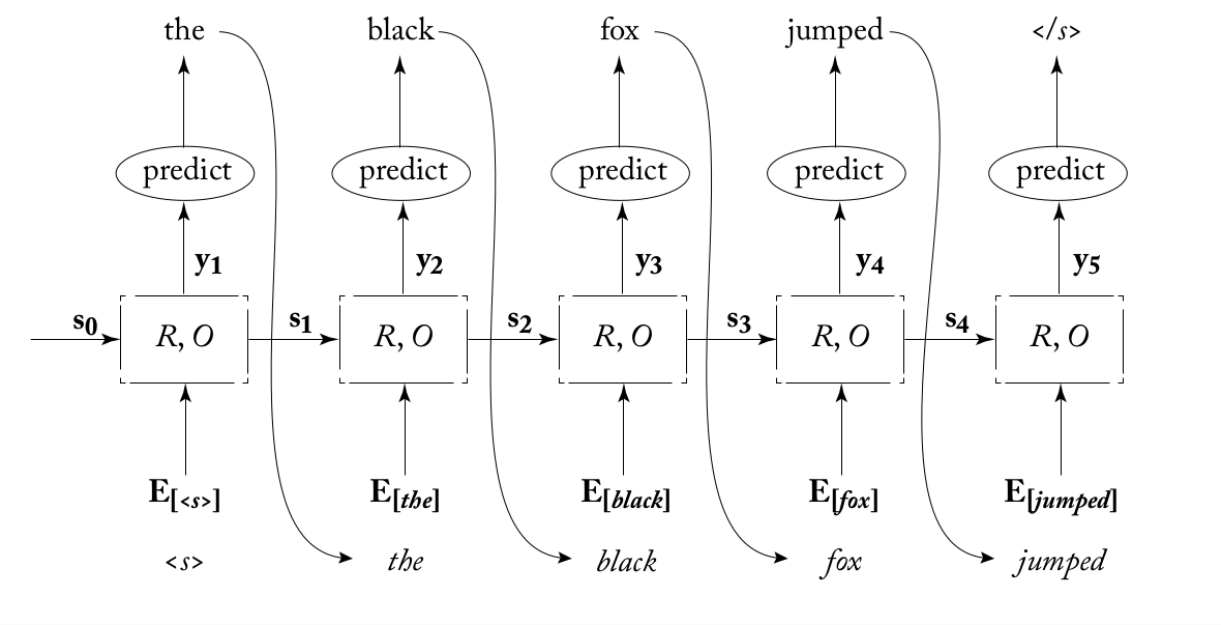
- The transducer pattern can be used for the sequence labeling task, such as NER or POS tagging: we take $\vec{x}_{i:n}$ to be feature representations for the n words of a sentence, and t_i as an input for predicting the tag assignment of word i based on words $1:i$.
    - For POS tagging, we want to classify each token as noun, verb, adverb, etc.
    - In NER, we want to identify and classify each token as belonging to a named entity (people, organizations, locations, etc), or no entity.
- We can also use this pattern for language modeling, where the sequence of words $x_{1:n}$ is used to predict a distribution over the $(i+1)th$ word.
    - Using RNNs as transducers allows us to relax the Markov assumption that is traditionally taken in language models and HMM taggers, and condition on the entire prediction history.

Language Models and Language Generation

- RNNs can be used to train language models by tying the output at time *i* with its input at time $i + 1$.
- This network can be used to generate sequences of words or random sentences.
- Generation process: predict a probability distribution over the first word conditioned on the start symbol, and draw a random word according to the predicted distribution.
- Then predict a probability distribution over the second word conditioned on the first, and so on, until predicting the end-of-sequence $< /\text{s}>$ symbol.

Transducer for Language Modeling

- After predicting a distribution over the next output symbols $P(t_i = k|t_{1:i-1})$, a token t_i is chosen and its corresponding embedding vector is fed as the input to the next step.



- Teacher-forcing: during **training** the generator is fed with the ground-truth previous word even if its own prediction put a small probability mass on it.
- It is likely that the generator would have generated a different word at this state in **test time**.

How to generate text?

- The decoder aims to generate the output sequence with maximal score (or maximal probability), i.e., such that $\sum_{i=1}^{n} P(\hat{t}_i|\hat{t}_{1:i-1})$ is maximized.
- The non-markovian nature of the RNN means that the probability function cannot be decomposed into factors that allow for exact search using standard dynamic programming.
- Exact search: finding the optimum sequence requires evaluating every possible sequence (computationally prohibitive).
- Thus, it only makes sense to solving the optimization problem above approximately.
- Greedy search: choose the highest scoring prediction (word) at each step.
- This may result in sub-optimal overall probability leading to prefixes that are followed by low-probability events.

Language generation: Greedy Search

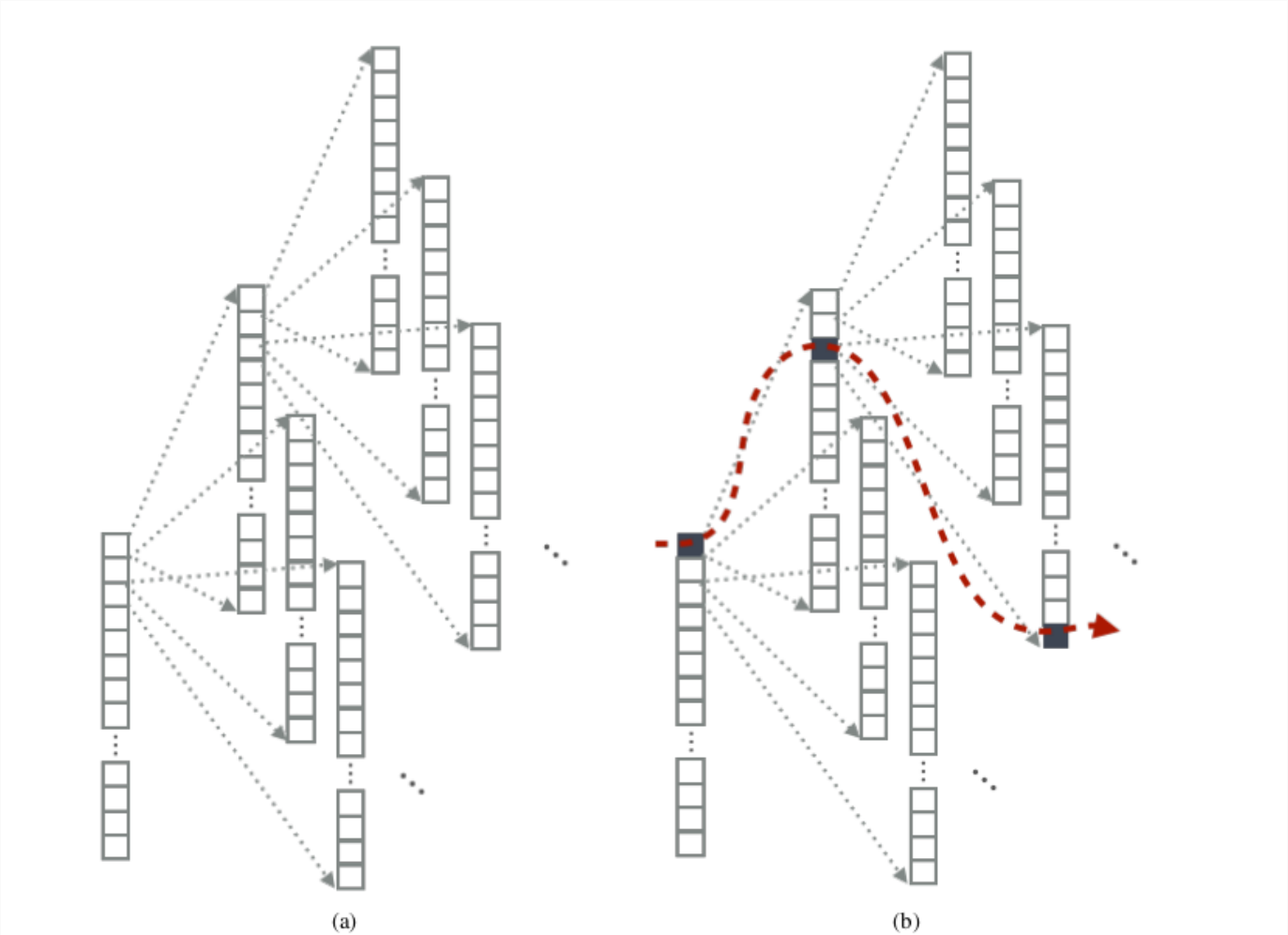


Figure 6.4: (a) Search space depicted as a tree. (b) Greedy search.

[0]Source: [Cho, 2015]

Language generation: Beam Search

- Beam search interpolates between the exact search and the greedy search by changing the size K of hypotheses maintained throughout the search procedure [Cho, 2015].
- The Beam search algorithm works in stages.
- We first pick the K starting words with the highest probability
- At each step, each candidate sequence is expanded with all possible next steps.
- Each candidate step is scored.
- The K sequences with the most likely probabilities are retained and all other candidates are pruned.
- The search process can halt for each candidate separately either by reaching a maximum length, by reaching an end-of-sequence token, or by reaching a threshold likelihood.
- The sentence with the highest overall probability is selected.

---

[0] More info at: `https://machinelearningmastery.com/beam-search-decoder-natural-language-processing/`

Language generation: Beam Search

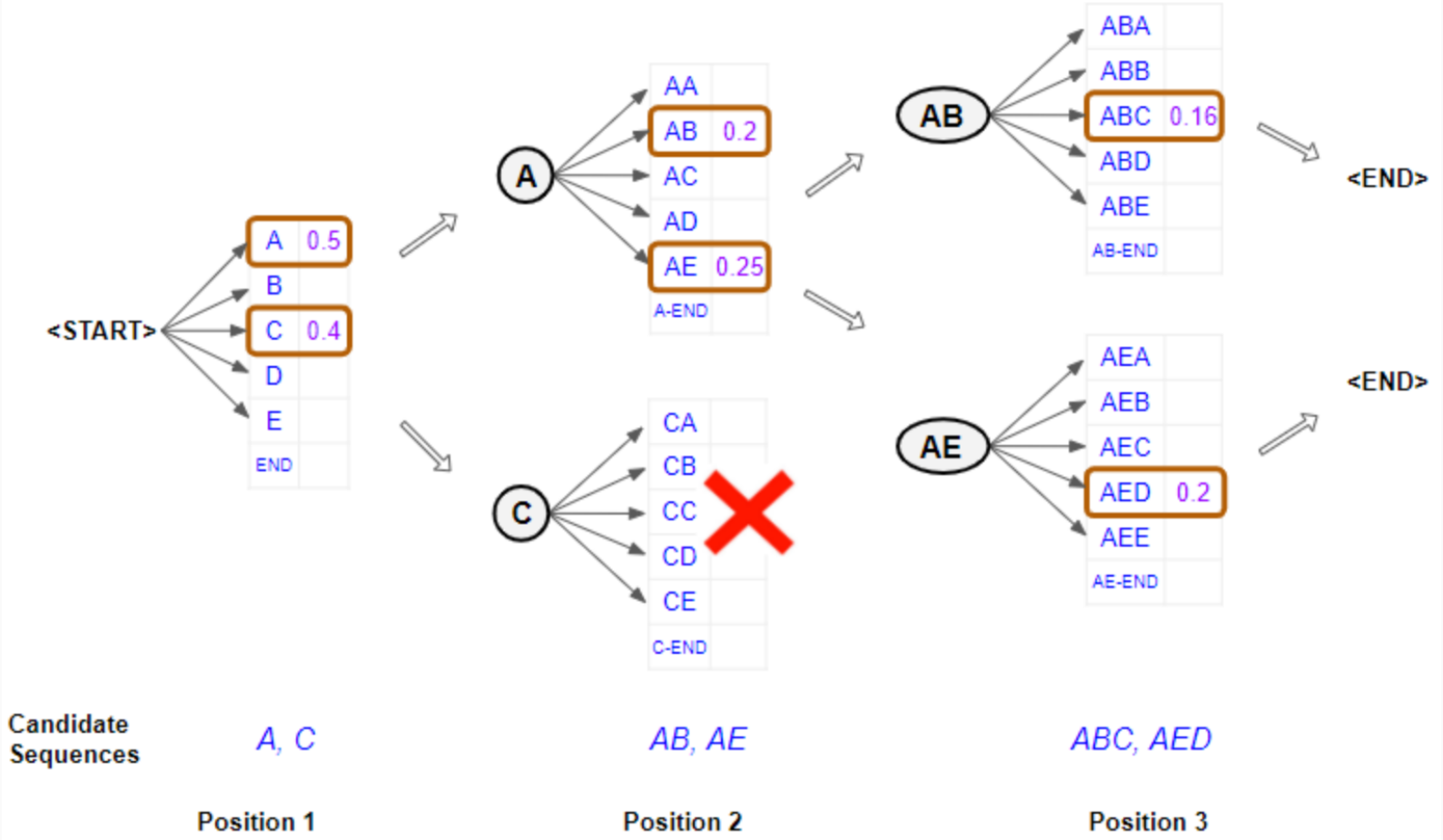


Figure 4: Beam Search. Source:
https://towardsdatascience.com/foundations-of-nlp-explained-visually-beam-search-how-it

Transducer: Language modeling

- The language modeling problem is then defined as:

$$p(t_{j+1} = k|\hat{t}_{1:j}) = f(O(s_{j+1}))$$
$$s_{j+1} = R(\hat{t}_j, s_j)$$
$$\hat{t}_j \sim p(t_j|\hat{t}_{1:j-1})$$

- f could be any function that maps the RNN state to a distribution over words. For instance, $f(\vec{x}) = \text{softmax}(\vec{x}W + \vec{b})$.

Transducer: Conditioned Generation

- The power of the RNN transducer is really revealed when moving to a conditioned generation framework.
- Suppose that we want to condition the generation of the text on the style of the user, and suppose that we have a context vector $\vec{c}$ that encodes that information.
- We can use that context vector to *condition* the generation of the text.
- We simply need to concatenate $\vec{c}$ to the input of the model.
- Then, the *conditioned* language model is defined by:

$$
\begin{aligned}
p(t_{j+1} = k|\hat{t}_{1:j}, c) &= f(O(s_{j+1})) \\
s_{j+1} &= R([\hat{t}_j : c], s_j) \\
\hat{t}_j &\sim p(t_j|\hat{t}_{1:j-1}, c)
\end{aligned}
$$

Transducer: conditioned generation

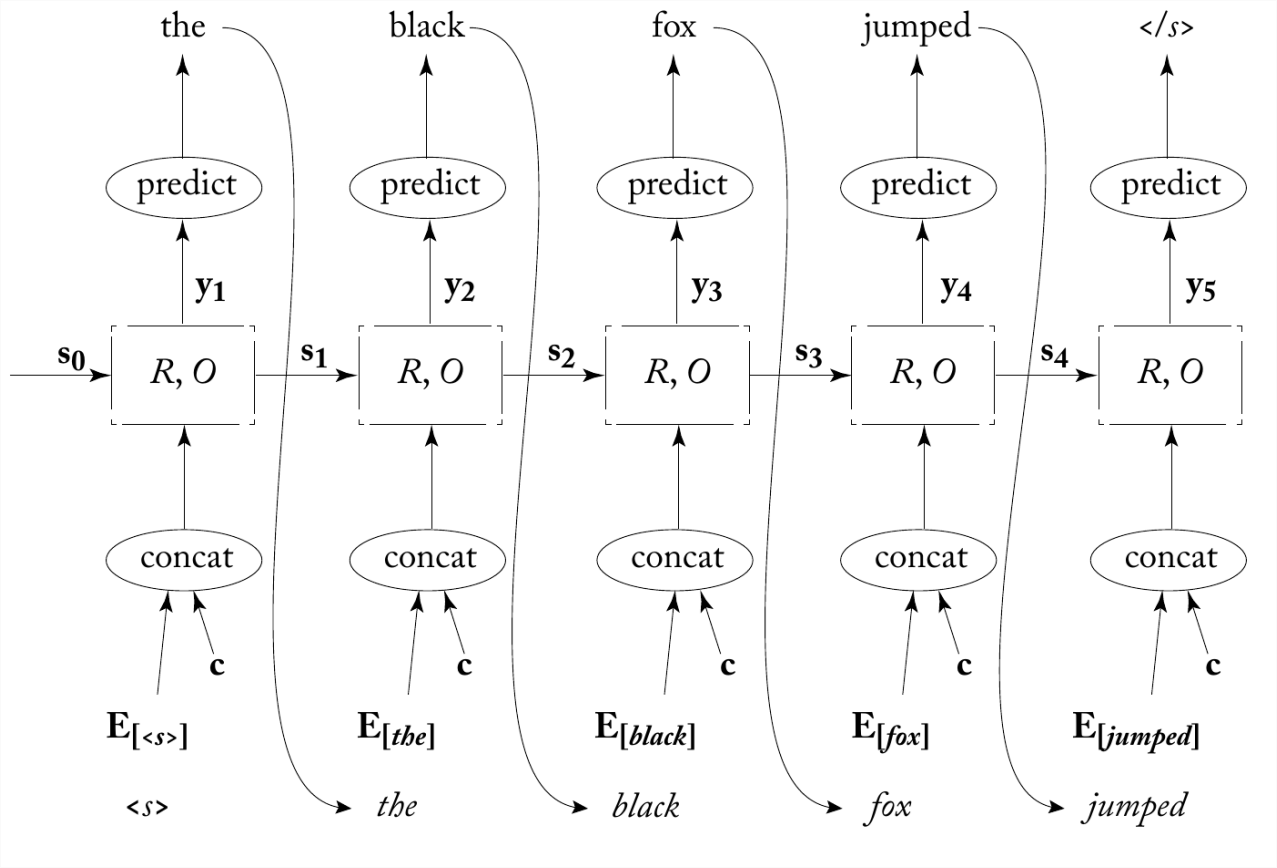


Figure 5: Conditioned RNN transducer generator

- What if we want to condition the generation on another sequence?

Sequence to Sequence

Sequence to Sequence Problems

- Until now, we've seen problems of the form:
  - $n \rightarrow 1$: text classification (e.g., sentiment classification, topic classification)
  - $n \rightarrow n$: sequence labeling (e.g., NER, POS tagging)
- But there are problems of the type $n \rightarrow m$: problems where the input sequence has length n and the output sequence has length m, with n and m not necessarily equal.
- These are called **sequence to sequence** problems.
- Nearly any task in NLP can be formulated as a sequence to sequence task.
  - Machine Translation: source language to target language.
  - Summarization: long text to short text.
  - Dialogue (chatbots): previous utterances to next utterance.

Sequence to sequence

- The core idea is to encode the input sequence as a vector $\vec{c}$, and use that vector as context for the conditioned generation.
- How can we encode the input sequence?
- Two RNNs: encoder and decoder (encoder-decoder framework)
- Encoder: One RNN is used to encode the source input into a vector $\vec{c}$.
- Decoder: Another RNN is used to decode the encoder's output and generate the target output.
- At each stage of the generation process the context vector $\vec{c}$ is concatenated to the input $\hat{t}_j$ and the concatenation is fed into the RNN.

Encoder Decoder Framework

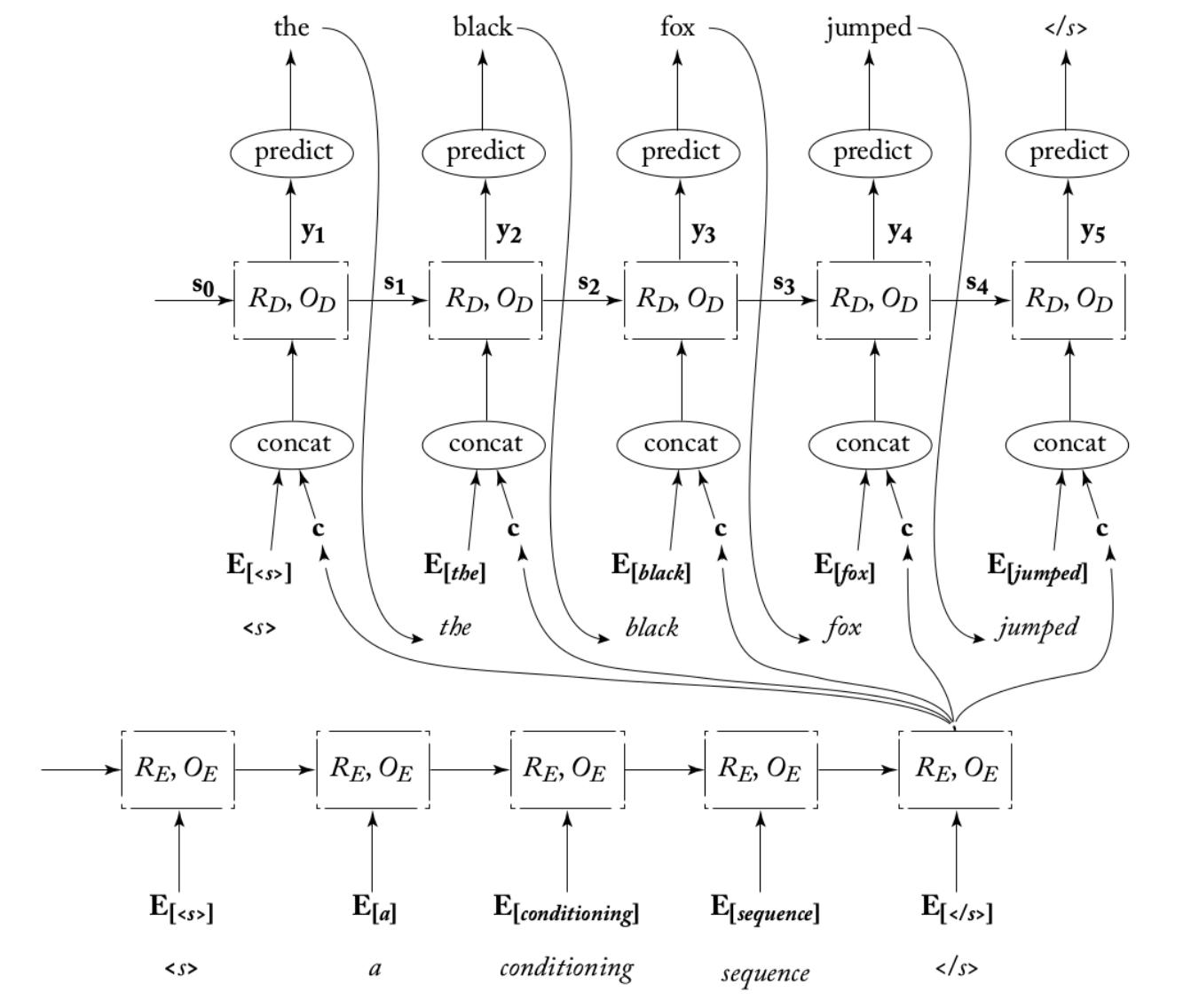
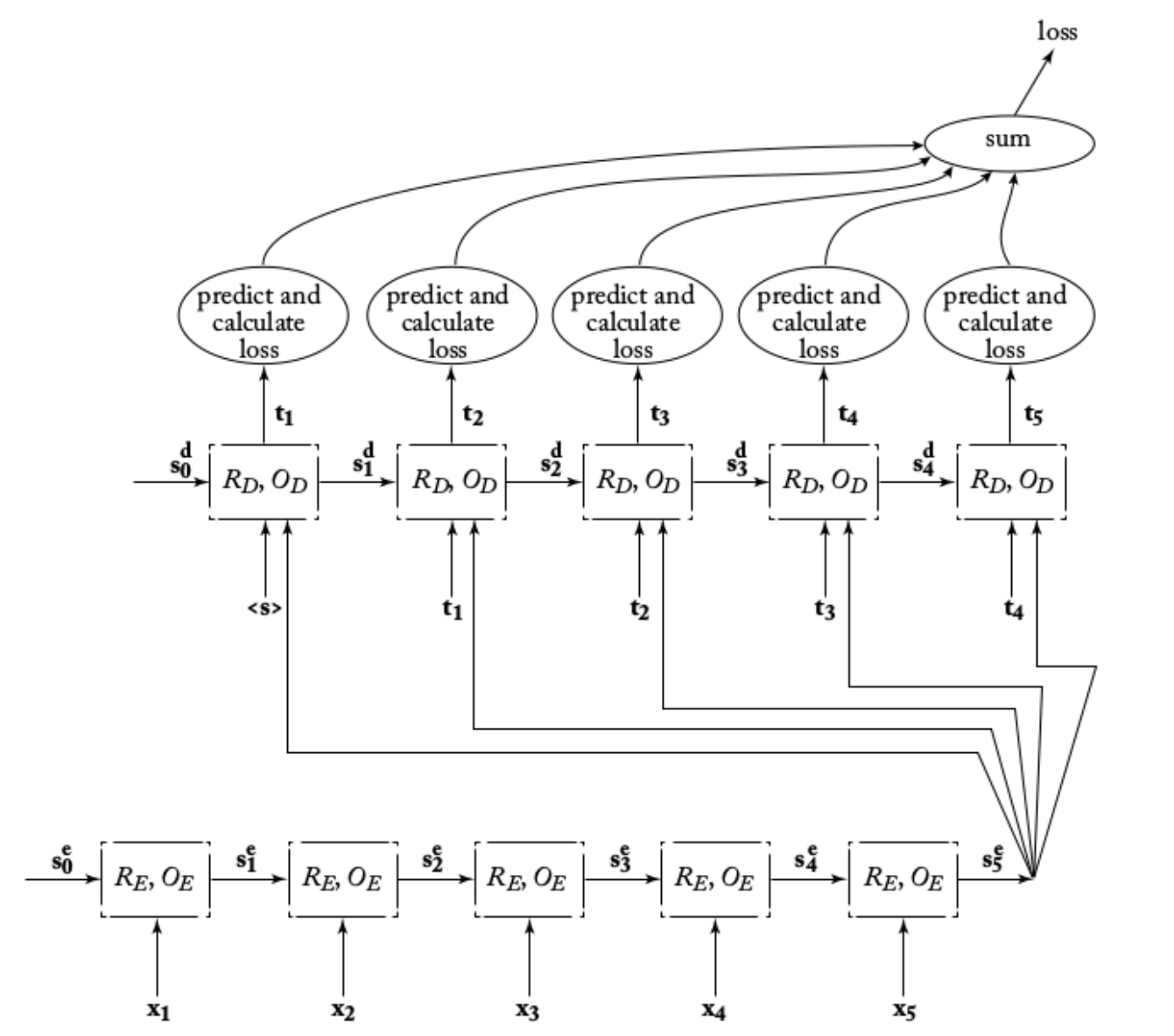


Figure 6: Sequence-to-sequence RNN generator

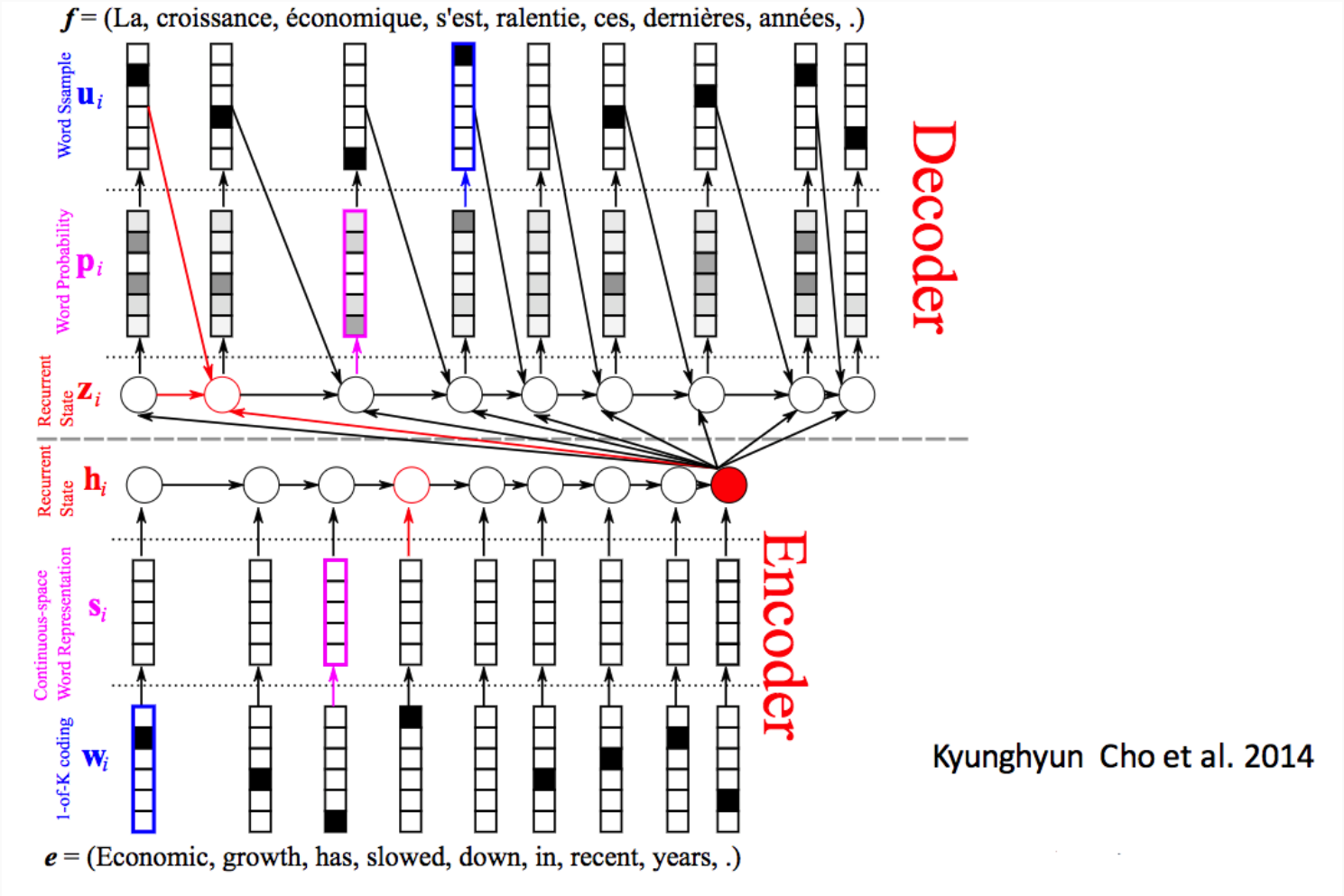
Encoder Decoder Framework

- This setup is useful for mapping sequences of length n to sequences of length m.
- The encoder summarizes the source sentence as a vector $\vec{c}$. Usually, $\vec{c}$ is the hidden state of the encoder after processing the last token of the source sentence.
- The decoder RNN is then used to predict (using a language modeling objective) the target sequence words conditioned on the previously predicted words as well as the encoded sentence $\vec{c}$.
- The encoder and decoder RNNs are trained jointly.
- The supervision happens only for the decoder RNN, but the gradients are propagated all the way back to the encoder RNN.

Sequence to Sequence Training Graph



Neural Machine Translation



Conditioned Generation: bottleneck

- In the encoder-decoder networks the input sentence is encoded into a single vector, which is then used as a conditioning context for an RNN-generator.
- This architecture forces the encoded vector $\vec{c}$ to contain all the information required for generation.
- It doesn't work well for long sentences!
- It also requires the generator to be able to extract this information from the fixed-length vector.
- "You can't cram the meaning of a whole %&!$# sentence into a single $&!#* vector!" - Raymond Mooney (ACL, 2014)
- This architecture can be can be substantially improved (in many cases) by the addition of an *attention mechanism*.

Attention

Encoder-decoder with attention

- Core idea: use a mechanism that allows the decoder to "look back" at the encoder's hidden states based on its current state.
- More concretely, the input $\vec{x}_{1:n}$ is encoded using a biRNN, producing n vectors $\vec{c}_{1:n} = \text{ENC}(\vec{x}_{1:n}) = \text{biRNN}^*(\vec{x}_{1:n})$.
- The generator (decoder) can use these vectors as a read-only memory representing the conditioning sequence.
- At any stage j of the generation process, it chooses which of the vectors $\vec{c}_{1:n}$ it should attend to, using a soft attention mechanism.
- This results in a focused context vector $\vec{c}^j = \text{attend}(\vec{c}_{1:n}, \hat{t}_{1:j})$.
- The generation process is then defined by

$$
\begin{aligned}
p(t_{j+1} = k | \hat{t}_{1:j}, \vec{x}_{1:n}) &= f(O(s_{j+1})) \\
s_{j+1} &= R([\hat{t}_j : \vec{c}^j], s_j) \\
\vec{c}^j &= \text{attend}(\vec{c}_{1:n}, \hat{t}_{1:j}) \\
\hat{t}_j &\sim p(t_j | \hat{t}_{1:j-1}, \vec{x}_{1:n})
\end{aligned}
$$

- What is *attend*$(\cdot, \cdot)$?

Attention mechanism

- Suppose we are at decoding step j, thus we have a decoder state $\vec{s}_j$. We want a mechanism that, given s_j, tells us how much we should focus on each vector $\vec{c}_i$.
- We want an scalar value ($\bar{\alpha}^j_{[i]}$), telling us "how important" is c_i at step j. These scalar values are called (unnormalized) *attention weights*.
- We can use any differentiable function returning a scalar out of two vectors $\vec{s}_j$ and $\vec{c}_i$.
- The simplest approach is a dot product: $\bar{\alpha}^j_{[i]} = \vec{s}_j \cdot \vec{c}_i$.
- The one we will use in these slides is Additive attention, which uses a Multilayer Perceptron: $\bar{\alpha}^j_{[i]} = MLP^{att}([\vec{s}_j; \vec{c}_i]) = \vec{v} \cdot \tanh([\vec{s}_j; \vec{c}_i]U + \vec{b})$

Attention mechanism

- We then normalize the attention weights so that all of them are positive and sum to one.
- $\alpha^j = \text{softmax}(\bar{\alpha}^j_{[1]}, \ldots, \bar{\alpha}^j_{[n]})$.
- Using the normalized attention weights, we can compute a weighted average of the vectors $\vec{c}_{1:n}$.
- Thus, the context the decoder sees in step j is

$$\vec{c}^j = \sum_{i=1}^{n} \vec{\alpha}^j_{[i]} \cdot \vec{c}_i$$

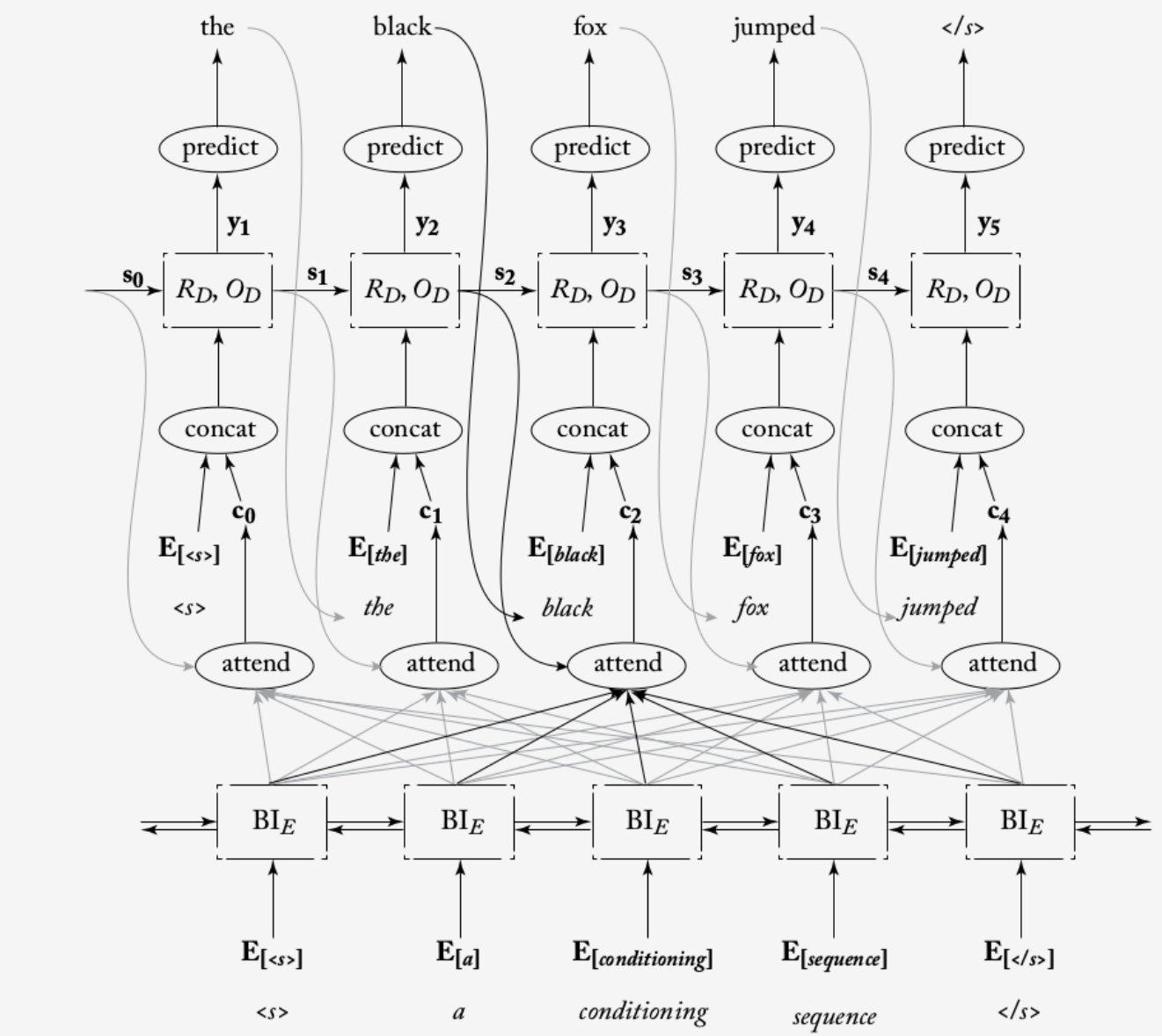
Attention mechanism

Combining everything:

$$\begin{aligned}
\text{attend}(\vec{c}_{1:n}, \hat{t}_{1:j}) &= c^j \\
c^j &= \sum_{i=1}^{n} \alpha^j_{[i]} \cdot \vec{c}_i \\
\alpha^j &= \text{softmax}(\bar{\alpha}^j_{[1]}, \ldots, \bar{\alpha}^j_{[n]}) \\
\bar{\alpha}^j_{[i]} &= \text{MLP}^{\text{att}}([\vec{s}_j : \vec{c}_i])
\end{aligned}$$

The encoder, decoder, and attention mechanism are all trained jointly in order to play well with each other.

Encoder-decoder with attention



Conditioned Generation with Attention

The entire sequence-to-sequence generation with attention is given by:

$$
\begin{aligned}
p(t_{j+1} = k | \hat{t}_{1:j}, \vec{x}_{1:n}) &= f(O_{\mathbf{dec}}(\vec{s}_{j+1})) \\
\vec{s}_{j+1} &= R_{\text{dec}}([\hat{t}_j : \vec{c}^j], \vec{s}_j) \\
\vec{c}^j &= \sum_{i=1}^{n} \alpha^j_{[i]} \cdot \vec{c}_i \\
\vec{c}_{1:n} &= \text{biRNN}^*_{\text{enc}}(\vec{x}_{1:n}) \\
\alpha^j &= \text{softmax}(\bar{\alpha}^j_{[1]}, \ldots, \bar{\alpha}^j_{[n]}) \\
\bar{\alpha}^j_{[i]} &= \text{MLP}^{\text{att}}([\vec{s}_j : \vec{c}_i]) \\
\hat{t}_j &\sim p(t_j | \hat{t}_{1:j-1}, \vec{x}_{1:n}) \\
f(z) &= \text{softmax}(\text{MLP}^{\text{out}}(z))
\end{aligned}
$$

Conditioned Generation with Attention

- Why use the biRNN encoder to translate the conditioning sequence $\vec{x}_{1:n}$ into the context vectors $\vec{c}_{1:n}$?
- Why we just don't attend directly on the inputs (word embeddings) $MLP^{att}([\vec{s}_j; \vec{x}_i])$?
- We could, but we get important benefits from the encoding process.
- First, the biRNN vectors $\vec{c}_i$ represent the items $\vec{x}_i$ in their sentential context.
- Sentential context: a window focused around the input item $\vec{x}_i$ and not the item itself.
- Second, by having a trainable encoding component that is trained jointly with the decoder, the encoder and decoder evolve together.
- Hence, the network can learn to encode relevant properties of the input that are useful for decoding, and that may not be present at the source sequence $\vec{x}_{1:n}$ directly.

Attention and Word Alignments

- In the context of machine translation, one can think of MLP^{att} as computing a soft alignment between the current decoder state $\vec{s}_j$ (capturing the recently produced foreign words) and each of the source sentence components $\vec{c}_i$.

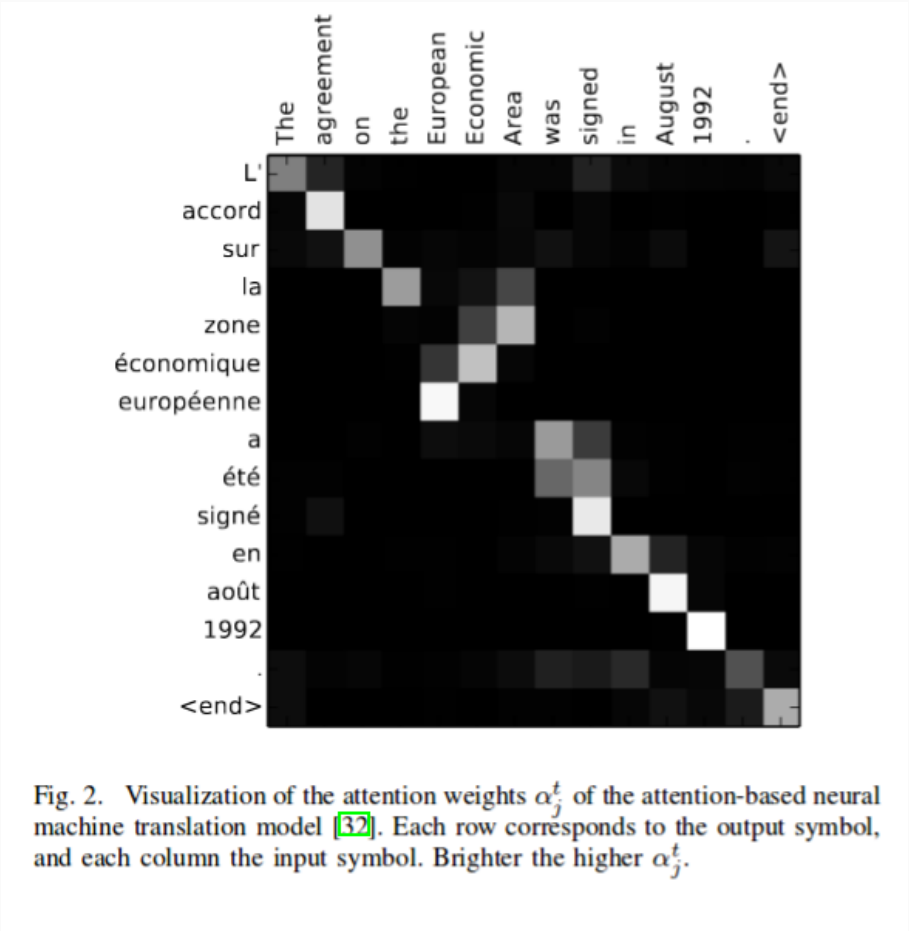


Fig. 2. Visualization of the attention weights α_j^t of the attention-based neural machine translation model [32]. Each row corresponds to the output symbol, and each column the input symbol. Brighter the higher α_j^t.

Figure 7: Source: [Cho et al., 2015]

Other types of Attention

Summary

Below is a summary table of several popular attention mechanisms (or broader categories of attention mechanisms).

Name	Alignment score function	Citation
Additive(*)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\boldsymbol{s}_t; \boldsymbol{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \boldsymbol{s}_t)$ Note: This simplifies the softmax alignment max to only depend on the target position.	Luong2015
General	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \mathbf{W}_a \boldsymbol{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \boldsymbol{h}_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \frac{\boldsymbol{s}_t^\top \boldsymbol{h}_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017
Self-Attention(&)	Relating different positions of the same input sequence. Theoretically the self-attention can adopt any score functions above, but just replace the target sequence with the same input sequence.	Cheng2016
Global/Soft	Attending to the entire input state space.	Xu2015
Local/Hard	Attending to the part of input state space; i.e. a patch of the input image.	Xu2015; Luong2015

(*) Referred to as "concat" in Luong, et al., 2015 and as "additive attention" in Vaswani, et al., 2017.
(^) It adds a scaling factor $1/\sqrt{n}$, motivated by the concern when the input is large, the softmax function may have an extremely small gradient, hard for efficient learning.
(&) Also, referred to as "intra-attention" in Cheng et al., 2016 and some other papers.

Figure 8: Source: `https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html`

Questions?

Thanks for your Attention!

(ba dum tss)

Source for most of these slides: [Goldberg, 2017], chapters 16 & 17.

References i


Cho, K. (2015).
Natural language understanding with distributed representation.
arXiv preprint arXiv:1511.07916.


Cho, K., Courville, A., and Bengio, Y. (2015).
Describing multimedia content using attention-based encoder-decoder networks.
IEEE Transactions on Multimedia, 17(11):1875–1886.

References ii

Goldberg, Y. (2017).
Neural network methods for natural language processing.
Synthesis Lectures on Human Language Technologies, 10(1):1–309.